RXVM: Reverse Engineering Amazon's Client-Side Behavioral Bot Detection System

Nicholas Kloster
Independent Researcher
United States
exilekingdom@proton.me

March 15, 2026

Abstract

We present a comprehensive reverse engineering of RXVM, Amazon's proprietary bytecode virtual machine embedded in every amazon.com product page. RXVM executes 15 obfuscated bytecode payloads in the browser to silently collect behavioral telemetry, fingerprint automation tools, and run a client-side neural network that scores each session as human or bot. We fully disassemble all 15 payloads, extract the complete architecture and weights of the A10 behavioral scoring model (a 31-input, three-layer neural network), decrypt the `rxc=` telemetry cookie, and document the AES-128-CBC and RC4 encryption pipeline used to exfiltrate data to Amazon's servers. We further identify a payload specifically targeting AI browsing agents, including a regular-expression scan for Perplexity AI's browser extension. Our analysis reveals that the RC4 encryption key for the `rxc=` cookie is shipped in plaintext within the cookie itself, making all collected telemetry trivially decryptable by any observer. We release a complete disassembler, cookie decryptor, and neural network reimplementation. This work raises significant questions about transparency, user consent, and the technical arms race between e-commerce platforms and automated browsing agents.

1 Introduction

Bot detection has become a critical concern for large e-commerce platforms. Amazon.com faces automated traffic from price scrapers, inventory bots, competitor intelligence tools, and AI shopping agents. To address this, Amazon has deployed a sophisticated client-side behavioral analysis system embedded directly in page JavaScript. Unlike traditional server-side bot detection, this system runs entirely in the visitor's browser, collecting fine-grained behavioral signals and scoring them with a neural network before the data ever reaches Amazon's servers.

We call this system RXVM, after the `rx` JavaScript object it operates through and the virtual machine architecture it employs. RXVM is notable for several reasons:

- It compiles detection logic to a *proprietary bytecode* that resists standard JavaScript analysis tools.
- It employs a *per-closure XOR key reseeding* scheme that defeats naive single-pass disassembly.
- It runs a *trained neural network* in the browser, scoring 31 behavioral and environmental features.

- It specifically detects and flags *AI browsing agents*, including Perplexity AI's browser extension.
- Despite AES-128-CBC encryption of server-bound telemetry, the `rxc=` cookie – which carries a subset of this telemetry – uses RC4 with a key shipped *in plaintext* within the cookie itself.

This paper makes the following contributions:

1. A complete specification of the RXVM bytecode instruction set, including opcode encoding, value types, and closure boundary semantics.
2. Full disassembly and annotation of all 15 payloads extracted from a live Amazon search results page.
3. Extraction and analysis of the A10 behavioral scoring model: its 31 input features, three-layer architecture, and complete weight set.
4. A working `rxc=` cookie decryptor demonstrating the weakness of the RC4 scheme.
5. Documentation of RXVM's AI agent detection capabilities targeting Perplexity AI and other browsing agents.
6. Discussion of the privacy and transparency implications of this architecture.

All tools are released as open source at the time of publication.

2 Background and Related Work

2.1 Client-Side Bot Detection

Bot detection has traditionally operated server-side, analyzing IP reputation, request rates, and HTTP header anomalies [**?**]. The shift to client-side detection reflects the limitations of server-side approaches against modern bots that faithfully replicate HTTP-level browser behavior.

Client-side detection leverages signals that are expensive to fake: mouse movement trajectories [**?**], keyboard dynamics [**?**], scroll behavior, and browser environment fingerprints. Academic work has shown that behavioral biometrics can distinguish humans from bots with high accuracy, and industry deployments by Cloudflare [**?**], PerimeterX [**?**], and HUMAN Security build on these findings.

2.2 JavaScript Obfuscation and Virtual Machines

JavaScript-based virtual machines are a well-documented obfuscation technique [**?**]. By compiling program logic to a proprietary bytecode that only runs inside an interpreter shipped with the page, operators make static analysis significantly harder. Standard reverse engineering tools like de4js, JStillery, and Prettier operate on JavaScript ASTs and are ineffective against binary bytecode formats. Prior work has analyzed similar VM-based protections in game anti-cheat systems [**?**] and DRM schemes [**?**], but e-commerce bot detection VMs have received less academic attention.

2.3 Behavioral Telemetry and Privacy

The collection of fine-grained behavioral telemetry without explicit user consent raises questions under privacy regulations including GDPR [**?**] and CCPA [**?**]. Previous work has documented the scope of JavaScript-based tracking on the web [**?**] and the gap between stated and actual data collection practices [**?**]. This work contributes a detailed technical account of one such system.

3 System Architecture Overview

RXVM consists of three components: a JavaScript interpreter shipped as plaintext in the page, a set of 15 base64-encoded bytecode payloads executed via `rx.ex64("...")`, and a matrix math module (`rx.M`) used by the neural network layer.

```
<!-- Embedded in every Amazon product/search page -->
<script>
  // rx interpreter (plaintext JS, ~2KB minified)
  (function(window) { window.rx = {}; ... })(window);

  // 15 bytecode payloads
  rx.ex64("UlgB...");  // payload [0]: crypto
  rx.ex64("UlgB...");  // payload [1]: statistics
  // ... 13 more payloads
</script>
```

Figure 1: RXVM embedding structure in Amazon page HTML

The 15 payloads implement a complete pipeline: encryption infrastructure, statistical primitives, device fingerprinting, behavioral event collection, neural network inference, and result exfiltration. Table **??** summarizes each payload's role.

Table 1: RXVM Payload Summary

ID	Function	Key Export	Bytes
0	Crypto layer	`rx.ep`, `rx.ep4`, `rx.h`	1344
1	Statistics library	`rx.asum`, `rx.aavg`, `rx.ef16`	871
2	Cookie fingerprint	`rx.fnp`, `rx.fnpb`	194
3	State persistence	`rx.gd`, `rx.sfe`	717
4	Bot fingerprinting	Bitmask via `rx.p(16)`	845
5	Mouse click timing	`mc_s`, `mc_m`, `mc_c`	615
6	Mouse movement	`ms_*`, `ma_*`	936
7	Scroll behavior	`sc_s`, `sc_m`, `sc_c`	631
8	Navigation timing	`nd`, `nd_l`, `sl`	360
9	Viewport/screen dims	`vh`, `vw`, `sh`, `sw`	319
10	ML runtime	`rx.ML` (Dense, BatchNorm)	1026
11	ML scoring (A10)	Score via `rx.p(48)`	2875
12	Interaction gate	`rx.tag('has-strong-interaction')`	673
13	Cookie action track	`rx.tag('rx-highest-action')`	304
14	AI agent detection	Perplexity regex scan	1193

4 The RXVM Bytecode Format

4.1 Payload Structure

Each payload is a base64-encoded binary blob. After decoding, the structure is:

```
Byte 0:    0x52 ('R')  -- magic
```

```
Byte 1:    0x58 ('X')  -- magic
Byte 2:    version     -- currently 0x01
Bytes 3+:  XOR-obfuscated opcode stream
```

4.2 XOR Obfuscation and Key Reseeding

Each byte is XOR'd with a key derived from its offset:

$$\text{key}(\text{offset}) = (\text{offset} \bmod 127) + 37$$

Critically, this formula is applied *per closure body*. When the top-level walk encounters a `MAKE_CLOSURE` or `MAKE_NAMED_CLOSURE` instruction, it records the closure's starting offset and jumps past the body. Each closure body is then disassembled separately with its key reseeded from *its own starting offset*, not the file offset. This means a disassembler that treats the entire blob as a single XOR stream will produce garbage for all but the first closure. This is a deliberate anti-analysis measure.

4.3 Instruction Set

The VM supports approximately 30 opcodes. We group them by category:

Table 2: RXVM Opcode Summary

Category	**Opcodes**	**Description**
Storage	`STORE_LOCAL`, `STORE_GLOBAL`, `PROP_SET`	Variable assignment
Control flow	`JUMP`, `JMP_TRUE`, `JMP_FALSE`	Signed 16-bit relative jumps
Functions	`MAKE_CLOSURE`, `MAKE_NAMED_CLOSURE`	Function creation with skip offset
Calls	`CALL_0`–`CALL_N`, `NEW_0`–`NEW_N`	Invocation (0 to N args)
Iteration	`ITER_INIT`, `ITER_NEXT`	For-of style iteration
Expressions	`PUSH`, `PUSH_ACC`, `NOT`	Stack management
Arithmetic	+, -, *, /, %, &, —, ˆ, <<, >>, >>>	Inline binary operators
Comparison	>, <, >=, <=, ==, ===, !=, !==, in	Inline comparison operators
Other	`LOAD_STRINGS`, `RESTORE_ARGS`, `RET`, `NOP`	Misc

4.4 Value Types

The bytecode encodes typed values inline with a type-tag prefix byte:

- **Constants**: `undefined`, `null`, `true`, `false`
- **Integers**: variable-length encoded (varint) or fixed i32
- **Floats**: IEEE 754 f32, or a custom 8-bit minifloat for compact weight storage
- **Strings**: inline UTF-8 or string table reference
- **Collections**: array and object literals with inline elements
- **References**: local slot (`L`n) or global slot (`G`n)

The 8-bit minifloat uses a 1+4+3 bit layout (sign, exponent with bias 7, mantissa), providing approximately 120 distinct positive values. This is used exclusively for neural network weight storage.

4.5 Disassembler Implementation

Our v4 disassembler (`rxvm_disasm_v4.py`) handles the closure reseeding problem through a two-phase approach:

1. **Phase 1 – Top-level walk**: Process the payload from offset 3, recording all `(body_addr, end_addr)` pairs from closure instructions but skipping over closure bodies.
2. **Phase 2 – Closure disassembly**: For each recorded closure, instantiate a new reader seeded at the closure's starting offset, then disassemble within the `[body_addr, end_addr)` boundary. Newly discovered nested closures are queued for processing.

5 The Crypto Layer – Payload [0]

Payload [0] implements the encryption pipeline that protects all outgoing telemetry. It defines eight closures:

5.1 Key Derivation

When both a session ID (`sid`) and request ID (`rid`) are available:

$$\text{AES key} = \texttt{importKey}(\text{SHA-256(sid), AES-128-CBC}) \tag{1}$$

$$\text{AES IV} = \text{SHA-256(rid)}[0{:}16] \tag{2}$$

The raw SHA-256 digest of `sid` is also stored as the RC4 fallback key.

5.2 Encryption Paths

Primary (AES-128-CBC): Both paths prepend a 4-byte little-endian Unix timestamp to the plaintext before encryption, providing server-side replay detection. The primary path uses `crypto.subtle.encrypt` on modern browsers.

Fallback (RC4): A textbook RC4 implementation is used when WebCrypto is unavailable. The implementation includes a complete KSA (key scheduling algorithm) and PRGA (pseudo-random generation algorithm), with an extracted array-swap helper since RXVM's bytecode lacks destructuring.

5.3 Custom Hash Function

A custom 32-bit hash (`rx.h`) is used for session and request ID hashing. It uses xxHash32's multiply constant (`0x9E3779B1`) in a per-byte loop followed by Murmur3's fmix32 avalanche, seeded with `0xDEADBEEF`:

```
function hash(input) {
  let h = 0xDEADBEEF;
  for (const b of new TextEncoder().encode(input))
```

```
    h = Math.imul(0x9E3779B1, h ^ b);
  h ^= input.length;
  h ^= h >>> 16; h = Math.imul(0x85EBCA6B, h);
  h ^= h >>> 13; h = Math.imul(0xC2B2AE35, h);
  h ^= h >>> 16;
  return h >>> 0;
}
```

6 Behavioral Data Collection – Payloads [1]–[9]

6.1 Statistics Library (Payload [1])

Payload [1] provides mathematical primitives used by all behavioral collectors: array sum, mean, standard deviation, and a byte-serialized summary function. Notably, it implements a complete IEEE 754 half-precision (float16) codec for compact neural network score transmission.

6.2 State Persistence and EMA Smoothing (Payloads [2]–[3])

Raw behavioral events are not transmitted directly. Instead, each new measurement updates an exponential moving average (EMA):

$$\mathrm{EMA}_t = \alpha \cdot x_t + (1 - \alpha) \cdot \mathrm{EMA}_{t-1}$$

with $\alpha = 0.5$ by default. The EMA state is persisted to `localStorage` as an RC4-encrypted JSON blob, keyed by `hash(userAgent + sessionId)`. This enables the model to maintain stable feature estimates across page loads and navigation events.

6.3 Bot Fingerprinting (Payload [4])

Payload [4] runs eight automation-detection checks, packing results into a bitmask:

Table 3: Bot Fingerprint Checks

Bit	Feature	Detection Method
0	Playwright	`window.playwright` presence
1	WebDriver	`navigator.webdriver` or `'webdriver' in window`
2	ChromeDriver	`Object.keys(window)` scan for `cdc_` prefix
3	Puppeteer	`window` scan for `_Array`/`_Symbol`/`_Proxy` polyfills
4	PhantomJS	`_phantom` or `callPhantom` presence
5	SwiftShader	WebGL unmasked renderer string contains "SwiftShader"
6	Headless dims	`outerHeight === innerHeight`
7	Headless body	`body.clientWidth === innerWidth`

6.4 Behavioral Collectors (Payloads [5]–[9])

Three collectors capture human interaction patterns:

Mouse Click Timing (Payload [5]): Records mousedown/mouseup duration for each click. After 15 samples, computes count/mean/sum statistics fed to the neural network as `mc_c`, `mc_m`, `mc_s`.

Mouse Movement (Payload [6]): Tracks pixel-per-second velocity and angular acceleration between consecutive `mousemove` events. Resets tracking state if inter-event gap exceeds 200ms (stale cursor detection). Produces six features after 255 samples: `ms_*` (velocity) and `ma_*` (acceleration).

Scroll Behavior (Payload [7]): Mirrors the mouse movement collector for `scrollY` deltas, producing `sc_c`, `sc_m`, `sc_s`.

Navigation Timing (Payload [8]): Tracks visit count and seconds since first visit via `localStorage`, producing `sl` (session length), `nd` (navigation duration), and log-transformed `nd_l`.

Viewport/Screen Dimensions (Payload [9]): Captures `innerWidth`/`innerHeight` and `screen.width`/ `screen.height` with log-transformed variants.

7 The A10 Neural Network – Payloads [10]–[11]

7.1 Architecture

The A10 model is a fully-connected feed-forward network running in the browser:

$$\mathbf{x} \xrightarrow{\text{Normalize}} \xrightarrow{\text{Dense}(31\to16)} \xrightarrow{\text{BN}} \xrightarrow{\text{ReLU}} \xrightarrow{\text{Dense}(16\to16)} \xrightarrow{\text{BN}} \xrightarrow{\text{ReLU}} \xrightarrow{\text{Dense}(16\to1)} \xrightarrow{\sigma} \hat{y} \in [0,1]$$

The output maps to: 0 = bot/suspicious, 1 = genuine human engagement.

7.2 Input Features

The model takes 31 features organized into six categories:

Table 4: A10 Model Input Features

Category	**Features**	**Count**	**Source**
Scroll behavior	`sc_s`, `sc_m`, `sc_c`	3	Payload [7]
Mouse velocity	`ms_s`, `ms_m`, `ms_c`	3	Payload [6]
Mouse acceleration	`ma_s`, `ma_m`, `ma_c`	3	Payload [6]
Click patterns	`mc_s`, `mc_m`, `mc_c`	3	Payload [5]
Bot fingerprint	`hdl_1` through `hdl_8`	8	Payload [4]
Viewport/Screen	`vh`, `vh_l`, `vw`, `vw_l`, etc.	8	Payload [9]
Page structure	`nd`, `nd_l`, `sl`	3	Payload [8]

7.3 Input Normalization

Each feature is normalized using five stored statistics per feature (mean, std, clip_max, clip_min, offset):

$$\tilde{x}_i = \frac{\text{clip}(x_i,\ \text{clip_min},\ \text{clip_max}) - \text{offset}_i}{\text{std}_i}$$

Non-finite values (NaN, Inf) are replaced with the feature mean before clipping. The normalization statistics reflect the distribution of real Amazon user sessions from the training population.

7.4 Layer Implementation

Dense layers: Standard matrix-vector multiply plus bias: $\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$, implemented via the `rx.M.mul` function in the RXVM core runtime.

BatchNorm layers: Running statistics (mean, variance) with learned scale/shift parameters (γ, β), $\epsilon = 0.001$:

$$y_i = \gamma_i \cdot \frac{x_i - \mu_i}{\sqrt{\sigma_i^2 + \epsilon}} + \beta_i$$

Weight encoding: Dense layer weights are stored as 8-bit minifloats (1+4+3 layout, exponent bias 7). The 31→16 layer's 496 weights occupy only 43 distinct magnitude levels.

7.5 Scoring Loop

A `setInterval` fires every 1100ms. On each tick, it checks `rx.sv` (state version counter). If no new behavioral data has arrived, the tick is skipped. Otherwise, the EMA state is read, features extracted, inference run, and the float16-encoded score transmitted via `rx.p(48)`.

7.6 Feature Importance

We analyze the A10 model's first-layer weight magnitudes to estimate feature importance. Behavioral signals (mouse velocity, acceleration, scroll) carry the highest weight magnitudes, consistent with the known difficulty of faking natural human movement. Bot fingerprint features provide strong binary signals for simple automation tools, while viewport/screen dimensions and page structure features provide contextual normalization.

The model architecture suggests it can flag sophisticated bots that pass all fingerprint checks but exhibit inhuman movement statistics – for example, Bézier-curve mouse emulation with unnaturally uniform angular acceleration.

8 The rxc Cookie

8.1 Wire Format

The `rxc=` cookie carries accumulated behavioral metrics RC4-encrypted. After base64 decoding:

```
Byte 0:     fnpv (version, currently 0)
Bytes 1-4:  fnpb (RC4 key -- shipped in plaintext)
Bytes 5+:   RC4 ciphertext
```

After RC4 decryption:

```
Bytes 0-3:  Unix timestamp (uint32 LE) -- replay protection
Bytes 4+:   Metric stream [metric_id, ...data_bytes] pairs
```

8.2 Key Derivation Weakness

The RC4 key (`fnpb`) is derived as:

$$\text{fnpb} = \text{hash}(\texttt{navigator.userAgent} \parallel \texttt{rx.sid})$$

split into 4 bytes (little-endian). Both inputs are observable by any network-level observer: the User-Agent header is sent in every request, and the session ID appears in Amazon's standard

analytics parameters. Consequently, the RC4 "encryption" provides only obfuscation against casual inspection, not meaningful confidentiality. Any party who can observe the cookie can also reconstruct the key.

8.3 Metric IDs

Known metric IDs from our analysis:

```
16 (0x10) = mouse click timing stats (payload [5])
32 (0x20) = mouse movement stats (payload [6])
33 (0x21) = mouse acceleration stats (payload [6])
34 (0x22) = scroll velocity stats (payload [7])
35 (0x23) = paint timing (payload [8])
48 (0x30) = neural network score (payload [11])
```

9 AI Agent Detection – Payload [14]

Payload [14] is one of the most distinctive findings of this analysis. It specifically targets AI browsing agents, with logic tailored to Perplexity AI's browser extension:

9.1 DOM Scanning

```
// Queries for Perplexity overlay elements
document.querySelectorAll('[id^="pplx-agent"]')
// Tests each element's ID against:
/^pplx-agent(?:-[0-9]+_[0-9]+)?-overlay(?:-...)/
```

9.2 CSS Scanning

In addition to DOM elements, the payload scans:

- All `<style>` elements' `textContent`
- All `cssRules` across all stylesheets

for the same Perplexity pattern, catching agents that inject styles but not visible DOM nodes.

9.3 Persistence

When detected, Amazon sets two cookies with randomized TTLs:

- `amzn-ctxv-id`: 6–7 days TTL for DOM detection
- `amzn-css-id`: 83–100 days TTL for CSS detection

Both are set with `Path=/; SameSite=Lax` and `Secure` on HTTPS. The randomized TTLs suggest deliberate obfuscation of the detection mechanism. A `setInterval` polls every 2000ms with a safety cap at 120 seconds, detecting agents that inject content after initial page load.

9.4 Implications

This payload demonstrates that Amazon is actively monitoring for AI-powered shopping assistants and making product decisions (pricing, inventory) that may treat AI-assisted and direct human sessions differently. The 83–100 day persistence window for CSS-detected agents suggests this information is used for long-term session profiling, not just immediate traffic shaping.

10 Privacy and Transparency Analysis

10.1 Data Collected

RXVM collects, at minimum, on every Amazon product or search page visit:

- Complete mouse trajectory statistics (velocity, acceleration, counts)
- All scroll behavior statistics
- Click timing patterns
- Browser automation tool presence (8-bit fingerprint)
- Viewport and screen dimensions (including log-transformed variants)
- Navigation duration and visit frequency (via `localStorage`)
- A neural network score classifying the session
- Presence of specific browser extensions (Perplexity AI)

10.2 Consent and Disclosure

Amazon's privacy policy, at the time of this research, describes the collection of "interaction information" including mouse clicks and scrolling. It does not disclose the existence of a client-side virtual machine, neural network inference, browser automation fingerprinting, or detection of specific named third-party browser extensions. The gap between the disclosed data collection practices and the technical reality documented here is substantial.

10.3 The rxc Cookie

The `rxc=` cookie's RC4 key is derived from public information and shipped in the cookie itself. Any third party who can observe the cookie – for example, Amazon's CDN providers, ISPs on unencrypted connections, or other first-party Amazon scripts – can decrypt its contents. While HTTPS protects the cookie from passive network observers, the fundamental design choice to ship the decryption key in plaintext alongside the ciphertext undermines the stated confidentiality of this telemetry channel.

11 Responsible Disclosure

Following standard responsible disclosure practices, we reported these findings to Amazon's security team via Amazon's Vulnerability Research Program on HackerOne prior to publication of this paper. We provided Amazon with 90 days to review and respond before public disclosure.

12 Discussion

12.1 Effectiveness of RXVM's Obfuscation

RXVM's per-closure XOR reseeding is an effective defense against casual static analysis. Standard JavaScript deobfuscation tools are entirely ineffective against binary bytecode. However, the scheme is not cryptographically secure – the XOR key derivation is fully deterministic and recoverable from the payload itself. A determined analyst with knowledge of the key derivation formula can disassemble the entire payload; the barrier is knowledge, not computational hardness.

12.2 Neural Network Approach

The A10 model represents a notable deployment of client-side ML for bot detection. Running inference in the browser rather than the server eliminates the latency of a round-trip for the scoring decision and makes the feature extraction harder to tamper with (since it runs in the same JavaScript context as the page). The 31-feature, 3-layer architecture is small enough to be practical on mobile devices.

The use of EMA smoothing for behavioral features is a particularly noteworthy design choice. By dampening individual event spikes, the model becomes more robust to adversarial manipulation of individual events while remaining sensitive to sustained behavioral patterns.

12.3 The AI Agent Detection Arms Race

Payload [14]'s targeting of Perplexity AI by name suggests that Amazon's detection capabilities are updated reactively as new AI browsing tools emerge. The 83–100 day cookie persistence for CSS-detected agents, combined with the polling behavior, suggests a system designed for sustained tracking of AI tool users rather than simple session blocking.

13 Related Tools Released

We release the following tools alongside this paper:

- **rxvm_disasm_v4.py**: Complete RXVM bytecode disassembler with closure reseeding, degenerate output detection, and annotated output.
- **rxc_decrypt.py**: `rxc=` cookie decryptor with hex dump and metric stream parser.
- **a10_model.py**: Python reimplementation of the A10 neural network with extracted weights, feature importance analysis, and simulated profile scoring.
- **extract_rx_runtime.py**: Utility to extract the `rx.M` matrix module from saved Amazon page HTML.

14 Conclusion

We have presented a comprehensive reverse engineering of RXVM, Amazon's client-side bot detection system. Our analysis reveals a sophisticated architecture combining bytecode obfuscation, multi-modal behavioral telemetry, neural network inference, and targeted AI agent detection – all running silently in the visitor's browser. We fully disassemble all 15 bytecode payloads, extract the complete A10 neural network, and demonstrate a weakness in the `rxc=` cookie's RC4 encryption scheme.

This work has implications beyond Amazon. Client-side VM-based bot detection is likely deployed by other large e-commerce platforms. The techniques documented here – per-closure key reseeding, behavioral EMA smoothing, hybrid AES/RC4 telemetry, and named AI agent detection – represent the current state of the art in commercial bot detection. Understanding these systems is essential for researchers studying web privacy, browser security, and the technical dynamics of the emerging AI browsing agent ecosystem.

References

[1] Wang, J. et al. (2018). *Botscope: Measuring Bot Behavior at Scale*. Proceedings of the Web Conference.

[2] Shen, C. et al. (2019). *Mouse Dynamics Based Authentication*. IEEE Transactions on Information Forensics and Security.

[3] Monaco, J. (2018). *Behavioral Biometrics for Continuous Authentication*. IEEE Security & Privacy.

[4] Cloudflare. (2020). *Bot Management: How Cloudflare detects bots*. Cloudflare Blog.

[5] PerimeterX. (2021). *Bot Defender Technical Overview*. PerimeterX Technical Documentation.

[6] Xu, W. et al. (2012). *Auror: Defending against poisoning attacks in collaborative deep learning systems*. ACSAC.

[7] Chen, J. et al. (2020). *Reverse Engineering Game Anti-Cheat Systems*. USENIX Security.

[8] Miramirkhani, N. et al. (2019). *Spotless Sandboxes: Evading Malware Analysis Systems using Wear-and-Tear Artifacts*. IEEE S&P.

[9] European Parliament. (2016). *General Data Protection Regulation*. Regulation (EU) 2016/679.

[10] California Legislature. (2018). *California Consumer Privacy Act*. Cal. Civ. Code §1798.100.

[11] Englehardt, S. & Narayanan, A. (2016). *Online Tracking: A 1-million-site Measurement and Analysis*. CCS 2016.

[12] Degeling, M. et al. (2019). *We Value Your Privacy ... Now Take Some Cookies*. NDSS 2019.